Course Title:HIGH PERFORMANCE COMPUTING

NAME : RAJ KUMAR GURRAPU

BATCH : 12

HALLTICKET : 2303A51782

Assignment 3: One-to-One

Communication Pattern using MPI

Scenario

In a distributed simulation, each process must exchange boundary data with its neighboring process.

Objective

Implement pairwise Send/Receive communication among MPI Processes.

```
nithish@LAPTOP-A2FOJ865: ~

  GNU nano 7.2                                                    pairwise_mpi.c *
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv); // Initialize MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get process rank
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total processes

    if (size % 2 != 0) {
        if (rank == 0) {
            printf("Please run with even number of processes.\n");
        }
        MPI_Finalize();
        return 0;
    }

    if (rank % 2 == 0) {
        data = rank * 10;
        printf("Process %d sending data %d to process %d\n", rank, data, rank + 1);
        MPI_Send(&data, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
    }

    else {
        MPI_Recv(&data, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received data %d from process %d\n", rank, data, rank - 1);
    }

    MPI_Finalize();
    return 0;
}

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line
```

Tasks

1. Run the program with an even number of processes.

```
nithish@LAPTOP-A2FOJ865:~$ nano pairwise_mpi.c
nithish@LAPTOP-A2FOJ865:~$ mpicc pairwise_mpi.c -o pairwise
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 2 ./pairwise mpirun -np 4 ./pairwise
Process 0 sending data 0 to process 1
Process 1 received data 0 from process 0
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 2 ./pairwise
mpirun -np 4 ./pairwise
Process 0 sending data 0 to process 1
Process 1 received data 0 from process 0
Process 0 sending data 0 to process 1
Process 1 received data 0 from process 0
Process 2 sending data 20 to process 3
Process 3 received data 20 from process 2
nithish@LAPTOP-A2FOJ865:~$
```

2. Each process with even rank sends data to the next odd-ranked process.

```
nithish@LAPTOP-A2FOJ865: ~
  GNU nano 7.2                                                    pairwise_mpi.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size % 2 != 0) {
        if (rank == 0)
            printf("Run with even number of processes\n");
        MPI_Finalize();
        return 0;
    }

    if (rank % 2 == 0) {
        data = rank * 10;
        printf("Process %d sending %d to %d\n", rank, data, rank + 1);
        MPI_Send(&data, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
    } else {
        MPI_Recv(&data, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received %d from %d\n", rank, data, rank - 1);
    }

    MPI_Finalize();
    return 0;
}
```

```
nithish@LAPTOP-A2FOJ865:~$ MPI_Comm_rank(MPI_COMM_WORLD, &rank);
-bash: syntax error near unexpected token `MPI_COMM_WORLD,'
nithish@LAPTOP-A2FOJ865:~$ nano pairwise_mpi.c
nithish@LAPTOP-A2FOJ865:~$ mpicc pairwise_mpi.c -o pairwise
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 4 ./pairwise
Process 2 sending 20 to 3
Process 0 sending 0 to 1
Process 1 received 0 from 0
Process 3 received 20 from 2
nithish@LAPTOP-A2FOJ865:~$
```

3. Odd-ranked processes receive and print the data.

nithish@LAPTOP-A2FOJ865: ~

GNU nano 7.2 pairwise_mpi.c *

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size, data;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size % 2 != 0) {
        if (rank == 0)
            printf("Please run with even number of processes\n");
        MPI_Finalize();
        return 0;
    }

    if (rank % 2 == 0) {
        data = rank * 10;
        printf("Process %d sending %d to %d\n", rank, data, rank + 1);
        MPI_Send(&data, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
    }


    else {
        MPI_Recv(&data, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process %d received %d from %d\n", rank, data, rank - 1);
    }

    MPI_Finalize();
    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

```
nithish@LAPTOP-A2FOJ865:~$ nano  pairwise_mpi.c
nithish@LAPTOP-A2FOJ865:~$ mpicc pairwise_mpi.c -o pairwise
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 2 ./pairwise
Process 1 received 0 from 0
Process 0 sending 0 to 1
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 4 ./pairwise
Process 1 received 0 from 0
Process 2 sending 20 to 3
Process 3 received 20 from 2
Process 0 sending 0 to 1
nithish@LAPTOP-A2FOJ865:~$ mpirun -np 8 ./pairwise
--------------------------------------------------------------------------
There are not enough slots available in the system to satisfy the 8
slots that were requested by the application:

  ./pairwise

Either request fewer slots for your application, or make more slots
available for use.

A "slot" is the Open MPI term for an allocatable unit where we can
launch a process.  The number of slots available are defined by the
environment in which Open MPI processes are run:

  1. Hostfile, via "slots=N" clauses (N defaults to number of
     processor cores if not provided)
  2. The --host command line parameter, via a ":N" suffix on the
     hostname (N defaults to 1 if not provided)
  3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
  4. If none of a hostfile, the --host command line parameter, or an
     RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number
of hardware threads instead of the number of processor cores, use the
--use-hwthread-cpus option.

Alternatively, you can use the --oversubscribe option to ignore the
number of available slots when deciding the number of processes to
launch.
--------------------------------------------------------------------------
nithish@LAPTOP-A2FOJ865:~$
```

4. Analyze blocking behavior of Send/Receive.

In this program, both MPI_Send() and MPI_Recv() are blocking operations. This means that the processes do not continue execution immediately—they wait until the communication is completed.

When an even-ranked process sends data, it pauses until the data is properly handled. Similarly, the odd-ranked process waits until it receives the data before continuing. Because of this, both sending and receiving processes stay synchronized during execution.

There is no deadlock in this program because the communication pattern is simple and well-organized. Even-ranked processes only send data, and odd-ranked processes only receive it. Since there is no circular waiting between processes, the program runs smoothly.

Overall, the blocking nature of MPI ensures that data is transferred correctly and that all processes remain properly coordinated.

Learning Outcomes

- **Pairwise communication patterns**
  I understood how processes communicate in pairs, where each process exchanges data directly with a specific neighboring process.
- **Blocking communication behavior**
  I learned that send and receive operations are blocking, meaning a process waits until the data transfer is completed before continuing execution.
- **Process synchronization**
  I understood how processes stay coordinated with each other during communication, ensuring correct and orderly data exchange.